

Typing Rules, Injectivity, and Subject Reduction for Dependent Type Theory with Π , Σ , $\mathbf{U}:\mathbf{U}$, and $\mathbf{Prop}:\mathbf{U}$

1 Judgments

We work with three mutually defined judgment forms:

$\Gamma \text{ ctx}$	Γ is a well-formed context
$\Gamma \vdash M : A$	M has type A in context Γ
$\Gamma \vdash M = N : A$	M and N are definitionally equal at type A

Contexts are snoc-lists: $\Gamma ::= \cdot \mid \Gamma, x:A$. We write $M\sigma$ for the result of applying a substitution σ to a term M , and $[t]$ for the substitution $\Gamma \rightarrow \Gamma, x:A$ extending the identity by t , so that $B[a]$ means $B(\text{id}, a)$.

2 Well-formed contexts

$$\frac{}{\cdot \text{ ctx}} \qquad \frac{\Gamma \vdash A : \mathbf{U}}{\Gamma, x:A \text{ ctx}}$$

3 Typing rules

Core rules

$$\frac{\text{VAR} \quad \Gamma \text{ ctx} \quad (x:A) \in \Gamma}{\Gamma \vdash x : A} \qquad \frac{\text{CONV} \quad \Gamma \vdash M : A \quad \Gamma \vdash A = B : \mathbf{U} \quad \Gamma \vdash B : \mathbf{U}}{\Gamma \vdash M : B} \qquad \frac{\text{U} \quad \Gamma \text{ ctx}}{\Gamma \vdash \mathbf{U} : \mathbf{U}}$$

Pi types

$$\frac{\text{PI} \quad \Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U}}{\Gamma \vdash \Pi(x:A)B : \mathbf{U}} \qquad \frac{\text{LAM} \quad \Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma, x:A \vdash M : B}{\Gamma \vdash \lambda(x:A)M : \Pi(x:A)B}$$

$$\frac{\text{APP} \quad \Gamma \vdash A : \mathbf{U} \quad \Gamma, x:A \vdash B : \mathbf{U} \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a : B[a]}$$

Sigma types

$$\frac{\text{SIGMA} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U}}{\Gamma \vdash \Sigma(x:A)B : \mathbb{U}} \quad \frac{\text{MKPAIR} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : B[M]}{\Gamma \vdash \text{pair}(M, N) : \Sigma(x:A)B}$$

$$\frac{\text{FST} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : \Sigma(x:A)B}{\Gamma \vdash \text{fst}(M) : A}$$

$$\frac{\text{SND} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : \Sigma(x:A)B}{\Gamma \vdash \text{snd}(M) : B[\text{fst}(M)]}$$

Propositions

$$\frac{\text{PROP} \quad \Gamma \text{ ctx}}{\Gamma \vdash \text{Prop} : \mathbb{U}} \quad \frac{\text{PROP-U} \quad \Gamma \vdash A : \text{Prop}}{\Gamma \vdash A : \mathbb{U}} \quad \frac{\text{PI-PROP} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \text{Prop}}{\Gamma \vdash \Pi(x:A)B : \text{Prop}}$$

4 Conversion rules

General

$$\frac{\text{REFL} \quad \Gamma \vdash M : A}{\Gamma \vdash M = M : A} \quad \frac{\text{SYM} \quad \Gamma \vdash M = N : A}{\Gamma \vdash N = M : A} \quad \frac{\text{TRANS} \quad \Gamma \vdash M = N : A \quad \Gamma \vdash N = P : A}{\Gamma \vdash M = P : A}$$

$$\frac{\text{CONV-EQ} \quad \Gamma \vdash M = N : A \quad \Gamma \vdash A = B : \mathbb{U} \quad \Gamma \vdash B : \mathbb{U}}{\Gamma \vdash M = N : B}$$

Pi computation and congruence

$$\frac{\text{BETA} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma, x:A \vdash M : B \quad \Gamma \vdash a : A}{\Gamma \vdash (\lambda(x:A)M) a = M[a] : B[a]}$$

$$\frac{\text{PI-CONG} \quad \Gamma \vdash A = A' : \mathbb{U} \quad \Gamma, x:A \vdash B = B' : \mathbb{U}}{\Gamma \vdash \Pi(x:A)B = \Pi(x:A')B' : \mathbb{U}}$$

$$\frac{\text{FUNEXT} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma, x:A \vdash f^+ x = g^+ x : B \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash g : \Pi(x:A)B}{\Gamma \vdash f = g : \Pi(x:A)B}$$

$$\frac{\text{APP-FUN} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash f = f' : \Pi(x:A)B \quad \Gamma \vdash a : A}{\Gamma \vdash f a = f' a : B[a]}$$

$$\frac{\text{APP-ARG} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash f : \Pi(x:A)B \quad \Gamma \vdash a = a' : A}{\Gamma \vdash f a = f a' : B[a]}$$

Sigma computation and congruence

$$\frac{\text{SIGMA-CONG} \quad \Gamma \vdash A = A' : \mathbb{U} \quad \Gamma, x:A \vdash B = B' : \mathbb{U}}{\Gamma \vdash \Sigma(x:A)B = \Sigma(x:A')B' : \mathbb{U}}$$

$$\frac{\text{BETA-FST} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : B[M]}{\Gamma \vdash \text{fst}(\text{pair}(M, N)) = M : A}$$

$$\frac{\text{BETA-SND} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : B[M]}{\Gamma \vdash \text{snd}(\text{pair}(M, N)) = N : B[M]}$$

$$\frac{\text{PAIR-ETA} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : \Sigma(x:A)B}{\Gamma \vdash \text{pair}(\text{fst}(M), \text{snd}(M)) = M : \Sigma(x:A)B}$$

$$\frac{\text{MKPAIR-FST} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M = M' : A \quad \Gamma \vdash N : B[M] \quad \Gamma \vdash N' : B[M']}{\Gamma \vdash \text{pair}(M, N) = \text{pair}(M', N') : \Sigma(x:A)B}$$

$$\frac{\text{MKPAIR-SND} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M : A \quad \Gamma \vdash N = N' : B[M]}{\Gamma \vdash \text{pair}(M, N) = \text{pair}(M, N') : \Sigma(x:A)B}$$

$$\frac{\text{FST-CONG} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M = M' : \Sigma(x:A)B}{\Gamma \vdash \text{fst}(M) = \text{fst}(M') : A}$$

$$\frac{\text{SND-CONG} \quad \Gamma \vdash A : \mathbb{U} \quad \Gamma, x:A \vdash B : \mathbb{U} \quad \Gamma \vdash M = M' : \Sigma(x:A)B}{\Gamma \vdash \text{snd}(M) = \text{snd}(M') : B[\text{fst}(M)]}$$

Propositions

$$\frac{\text{PROP-IRR} \quad \Gamma \vdash A : \text{Prop} \quad \Gamma \vdash M : A \quad \Gamma \vdash N : A}{\Gamma \vdash M = N : A}$$

$$\frac{\text{PROP-U-EQ} \quad \Gamma \vdash M = N : \text{Prop}}{\Gamma \vdash M = N : \mathbb{U}}$$

$$\frac{\text{PI-PROP-CONG} \quad \Gamma \vdash A = A' : \mathbb{U} \quad \Gamma, x:A \vdash B = B' : \text{Prop}}{\Gamma \vdash \Pi(x:A)B = \Pi(x:A')B' : \text{Prop}}$$

5 Head reduction

We define single-step head reduction $M \rightsquigarrow_1 N$ as the compatible closure of β -reduction at the head, extended with projection reductions for Sigma types:

$$\begin{array}{c}
\text{HR-BETA} \\
\hline
(\lambda(x:A)M) a \rightsquigarrow_1 M[a]
\end{array}
\quad
\begin{array}{c}
\text{HR-APP} \\
\hline
M \rightsquigarrow_1 M' \\
\hline
M a \rightsquigarrow_1 M' a
\end{array}
\quad
\begin{array}{c}
\text{HR-BETA-FST} \\
\hline
\text{fst}(\text{pair}(M, N)) \rightsquigarrow_1 M
\end{array}
\quad
\begin{array}{c}
\text{HR-BETA-SND} \\
\hline
\text{snd}(\text{pair}(M, N)) \rightsquigarrow_1 N
\end{array}$$

$$\begin{array}{c}
\text{HR-FST} \\
\hline
M \rightsquigarrow_1 M' \\
\hline
\text{fst}(M) \rightsquigarrow_1 \text{fst}(M')
\end{array}
\quad
\begin{array}{c}
\text{HR-SND} \\
\hline
M \rightsquigarrow_1 M' \\
\hline
\text{snd}(M) \rightsquigarrow_1 \text{snd}(M')
\end{array}$$

We write $M \rightsquigarrow^* N$ for the reflexive-transitive closure of $\rightsquigarrow_1$. As in the Pi-only system, the validity relation uses a wrapper $\text{Red } \Gamma M N A$ with phantom context and type parameters.

6 Pi injectivity

Theorem 1 (Pi injectivity — Corollary 6). *If $\Gamma \vdash A_0 = \Pi(x:B_1)F_1 : \mathbb{U}$, then there exist B_0, F_0 such that*

1. $A_0 \rightsquigarrow^* \Pi(x:B_0)F_0$,
2. $\Gamma \vdash B_0 = B_1 : \mathbb{U}$,
3. $\Gamma, x:B_0 \vdash F_0 = F_1 : \mathbb{U}$.

Corollary 2 (Pi–Pi injectivity). *If $\Gamma \vdash \Pi(x:A_0)B_0 = \Pi(x:A_1)B_1 : \mathbb{U}$, then*

$$\Gamma \vdash A_0 = A_1 : \mathbb{U} \quad \text{and} \quad \Gamma, x:A_0 \vdash B_0 = B_1 : \mathbb{U}.$$

Proof outline. The proof proceeds by domain-theoretic adequacy:

1. From the conversion, extract $\Gamma \vdash A_0 : \mathbb{U}$ by presupposition.
2. Apply conversion soundness at the bottom environment $\rho_\perp$ to transfer $\text{EvalRel}(\Pi B_1 F_1, \rho_\perp, \text{Pi}(\perp, \text{nil}))$ to $\text{EvalRel}(A_0, \rho_\perp, \text{Pi}(\perp, \text{nil}))$.
3. Apply bundled equality adequacy (`adequacyEqSub2`) to obtain `EqVal2` at the `Pi( $\perp$ , nil)` code, which unfolds to `EqValTyPi2` containing the head reduction and domain/codomain conversions.
4. Since Π is a head normal form, reduction uniqueness (`Red-unique-Pi`) gives exact equalities.

□

7 Sigma injectivity

Theorem 3 (Sigma injectivity). *If $\Gamma \vdash A_0 = \Sigma(x:B_1)F_1 : \mathbb{U}$, then there exist B_0, F_0 such that*

1. $A_0 \rightsquigarrow^* \Sigma(x:B_0)F_0$,
2. $\Gamma \vdash B_0 = B_1 : \mathbb{U}$,

3. $\Gamma, x:B_0 \vdash F_0 = F_1 : \mathbf{U}$.

Corollary 4 (Sigma-Sigma injectivity). *If $\Gamma \vdash \Sigma(x:A_0)B_0 = \Sigma(x:A_1)B_1 : \mathbf{U}$, then*

$$\Gamma \vdash A_0 = A_1 : \mathbf{U} \quad \text{and} \quad \Gamma, x:A_0 \vdash B_0 = B_1 : \mathbf{U}.$$

Proof outline. The proof follows the same strategy as Pi injectivity:

1. Extract $\Gamma \vdash A_0 : \mathbf{U}$ by presupposition.
2. Transfer $\text{EvalRel}(\Sigma B_1 F_1, \rho_\perp, \text{Sigma}(\perp, \text{nil}))$ to $\text{EvalRel}(A_0, \rho_\perp, \text{Sigma}(\perp, \text{nil}))$ via conversion soundness.
3. Apply bundled equality adequacy to obtain EqValTySigma2 containing head reduction and domain/codomain conversions.
4. Since Σ is a head normal form, reduction uniqueness (Red-unique-Sigma) gives exact equalities.

□

8 Subject reduction

As an application of Pi and Sigma injectivity, we prove subject reduction for single-step head reduction. The proof requires two mutually defined theorems due to the Snd case.

Theorem 5 (Subject reduction and subject conversion). *The following are proved by mutual induction on the typing derivation:*

1. **Subject reduction.** *If $\Gamma \vdash M : A$ and $M \rightsquigarrow_1 N$, then $\Gamma \vdash N : A$.*
2. **Subject conversion.** *If $\Gamma \vdash M : A$ and $M \rightsquigarrow_1 N$, then $\Gamma \vdash M = N : A$.*

The mutual definition is necessary because of the SND case: when $M \rightsquigarrow_1 M'$ and we have $\Gamma \vdash \text{snd}(M) : B[\text{fst}(M)]$, subject reduction for $\text{snd}(M) \rightsquigarrow_1 \text{snd}(M')$ must produce $\Gamma \vdash \text{snd}(M') : B[\text{fst}(M)]$. But the direct application of SND gives type $B[\text{fst}(M')]$. To bridge the gap, we need $\Gamma \vdash \text{fst}(M) = \text{fst}(M') : A$, which requires subject conversion (not just subject reduction) applied to M .

Proof outline. By mutual induction on the typing derivation, case-splitting on the head reduction step.

Subject reduction cases:

- CONV and PROP-U: peel off and recurse.
- APP with HR-APP ($f \rightsquigarrow_1 f'$): induction gives $\Gamma \vdash f' : \Pi(x:A)B$; rebuild with APP.
- APP with HR-BETA ($(\lambda(x:A')M') a \rightsquigarrow_1 M'[a]$): the key case. Use Pi injectivity to extract the body typing $\Gamma, x:A_0 \vdash M' : B_0$ from the lambda derivation (walking through CONV layers), then apply substitution. The helper lemma **ty-Lam-body** performs this extraction.
- FST with HR-FST ($M \rightsquigarrow_1 M'$): induction gives $\Gamma \vdash M' : \Sigma(x:A)B$; rebuild with FST.

- FST with HR-BETA-FST ($\text{fst}(\text{pair}(M, N)) \rightsquigarrow_1 M$): use Sigma injectivity to extract $\Gamma \vdash M : A$ from the MkPair derivation. The helper `ty-MkPair-components` performs this extraction.
- SND with HR-SND ($M \rightsquigarrow_1 M'$): induction on subject conversion gives $\Gamma \vdash M = M' : \Sigma(x:A)B$, hence $\Gamma \vdash \text{fst}(M) = \text{fst}(M') : A$ by FST-CONG. Subject reduction gives $\Gamma \vdash \text{snd}(M') : B[\text{fst}(M')]$; type conversion via $\text{fst}(M) = \text{fst}(M')$ yields $\Gamma \vdash \text{snd}(M') : B[\text{fst}(M)]$.
- SND with HR-BETA-SND ($\text{snd}(\text{pair}(M, N)) \rightsquigarrow_1 N$): use Sigma injectivity and `ty-MkPair-components` to extract $\Gamma \vdash N : B[M]$, then convert using $\Gamma \vdash M = \text{fst}(\text{pair}(M, N)) : A$ (the symmetric Beta-Fst).

Subject conversion follows from subject reduction: given $\Gamma \vdash M : A$ and $M \rightsquigarrow_1 N$, subject reduction gives $\Gamma \vdash N : A$, and the corresponding conversion rule for the head reduction step (β , β -fst, β -snd, or congruence) gives $\Gamma \vdash M = N : A$.

Key helper lemmas:

- `ty-Lam-body`: If $\Gamma \vdash \lambda(x:A')M : \Pi(x:A)B$, then $\Gamma, x:A \vdash M : B$. Proved by walking through CONV (using Pi injectivity) and eliminating PROP-U (since U cannot head-reduce to Π).
- `ty-MkPair-components`: If $\Gamma \vdash \text{pair}(M, N) : \Sigma(x:A)B$, then $\Gamma \vdash M : A$ and $\Gamma \vdash N : B[M]$. Proved by walking through CONV (using Sigma injectivity) and eliminating PROP-U.

□

9 Agda formalization

The entire development is formalised in Agda with flags `--without-K` `--exact-split` `--type-in-type` and `0 postulates` throughout.

Core files:

- `RawSyntax.agda` — raw terms (Π , Σ , λ , App, MkPair, Fst, Snd, U, Prop), substitution, weakening.
- `TypingRules.agda` — typing and conversion judgments.
- `PaperSemantics.agda` — trusted kernel: finite elements (`FinEl`, `FinFun`), information ordering, coherence, finite membership. No postulates.
- `RawSemantics.agda` — selection-based evaluation relation. No postulates.
- `TypingSemantics.agda` — typing and conversion soundness. 0 postulates.
- `Reduction.agda` — head reduction (`HeadRed1`, `Red`).

Validity stack (Validity5 series):

- `Validity5Core.agda` — base definitions of `Val2`/`EqVal2` with record types.
- `Validity5DownUpRestrict.agda` — monotonicity: downward, upward, and restriction in the code argument.
- `Validity5Fwd.agda` — forward lemmas.

- `Validity5SymTrans.agda` — symmetry and transitivity for `EqVal2`.
- `Validity5Sup.agda` — sup closure.
- `Validity5Lemmas.agda` — collected utility lemmas.

Main results:

- `Adequacy5.agda` + `Adequacy5Cases.agda` — adequacy proof (two-substitution form). 0 postulates.
- `Adequacy5Helpers.agda`, `Adequacy5HeadRed.agda` — helper lemmas for adequacy.
- `Injectivity5.agda` — Pi and Sigma injectivity. 0 postulates.
- `SubjectReduction5.agda` — subject reduction + subject conversion (mutual). 0 postulates.

References

- [1] T. Coquand and S. Huber, “An adequacy theorem for dependent type theory,” *Archive for Mathematical Logic*, vol. 57, pp. 649–676, 2018.